

Uno TP backend API

Every call the Uno TP web app makes for its data · from docs/backend-api.md · 29 September 2026

This document lists every call the Uno TP web app makes for its data. The app never connects to a database. It reads and writes everything through the backend API described here, plus a few outside services for document checks. The client code is in `src/UnoTP.Backend`. `BackendClient.cs` holds the backend routes, and there is one file per outside service in `External/` and `Idfy/`.

These routes are the ones the app proposes. If the backend's routes differ, change `BackendClient.cs`: the pages depend only on the interfaces.

Pages

Every page is under `/unotp`, in lowercase. The application a step works on is in its address, never in the session.

Address	Page
<code>/unotp</code>	Dashboard
<code>/unotp/new</code>	Investor Identification: search, drafts, and a new application
<code>/unotp/applications</code>	View Application: the partner's applications
<code>/unotp/applications/{appNo}/documents</code>	Upload Documents
<code>/unotp/applications/{appNo}/investor</code>	Investor Information
<code>/unotp/applications/{appNo}/payment</code>	Bank Details & Payment
<code>/unotp/applications/{appNo}/deposit</code>	FD Configuration
<code>/unotp/applications/{appNo}/review</code>	Review Summary
<code>/unotp/applications/{appNo}/submitted</code>	Application Submitted
<code>/unotp/pay-in-slips</code>	PIS Generation - Axis
<code>/unotp/links</code>	Short URL
<code>/unotp/admin</code>	Console Admin
<code>/unotp/entry</code> , <code>/unotp/logout</code> , <code>/unotp/session-expired</code> , <code>/unotp/unauthorized</code> , <code>/unotp/error</code>	Entry and status pages

Each page is one controller (`src/UnoTP/Controllers/{Page}Controller.cs`), one view model (`ViewModels/{Page}ViewModel.cs`) and one folder of views (`Views/{Page}/`), named after the page: `NewApplication`, `Documents`, `Investor`, `Payment`, `Deposit`, `Review`, `Submitted`, `Applications`, `PayInSlips`, `Links`, `Admin`, `Dashboard`, `Entry`. The plumbing (session, feature gate, errors, cache, app addresses) is in `Infrastructure/`.

A form posts to its page's address and a verb (`.../documents/upload`, `.../review/submit`). The old addresses (`/Home`, `/Dashboard`, `/Purchase/InvestorIdentification`, `/Apps/UnoTp/...`) redirect to these, with their query.

Configuration

Setting	Meaning
<code>Backend:BaseUrl</code>	The backend API. If blank, the app runs on the in-memory mock (<code>src/UnoTP.Backend.Mock</code>).
<code>Backend:TimeoutSeconds</code>	Default 30.
<code>Backend:External:{NsdL,Identify,Masking,Ocr,Verification,PanAadhaarLink,FaceMatch,Decrypt}</code>	Optional address for each outside service. The default is <code>{Backend:BaseUrl}/external/{name}/</code> , which means the backend proxies it.
<code>Idfy:BaseUrl</code>	<code>Idfy.Api</code> . When set, <code>Idfy</code> handles the checks it has an endpoint for (see below).
<code>Idfy:TimeoutSeconds</code>	Default 75. The <code>Idfy.Api</code> guide asks for at least 70.
<code>Entry:DemoUserId</code> , <code>Entry:DemoSysCode</code>	The user the demo comes in as when the app is opened without the portal's values. Used only while <code>Features:DemoData</code> is on; leave empty in production.
<code>Backend:ReferenceCacheMinutes</code>	Minutes the lists and rules (<code>GET reference</code> , <code>GET config</code>) are kept for. Default 10; 0 asks every time.

In the environment, use a double underscore, for example `Backend__BaseUrl` .

What the app keeps, and for how long

Answers that change seldom are kept in the app's memory (`src/UnoTP/Infrastructure/CachedBackend.cs` and `Lookups.cs`), so a page load does not ask the backend for them each time. Nothing that was not found is kept, and no failure is: the next request asks again.

Answer	Kept for	Notes
<code>GET reference</code> , <code>GET config</code> (every list and rule)	<code>Backend:ReferenceCacheMinutes</code> , default 10	The same for every partner
<code>GET sourcing/brokers</code> , <code>GET sourcing/staff</code>	<code>Backend:ReferenceCacheMinutes</code>	The same for every partner
<code>GET ifsc/{code}</code>	1 hour	Only a branch that was found; a new IFSC is found at once
<code>GET ifsc?q=</code> (the bank search)	5 minutes, and 1 minute in the browser	Only a search that found something; a new bank is found by the next search
<code>POST deposits/quote</code>	1 minute, never past the day	Keyed by amount, tenure, payout and category - nothing of the investor's
<code>GET console/schedule</code>	30 seconds	Dropped the moment the app adds, ends or removes a window or notice
<code>GET me</code>	5 minutes	Per user and per session; a new session asks again
<code>GET demo/*</code>	<code>Backend:ReferenceCacheMinutes</code>	

A backend that changes one of these and needs it seen sooner should say so; the times are set in `CachedBackend.cs` . The cache holds at most 50,000 entries.

Common to every backend call

- **Partner:** every request carries `X-Partner-Id`, the user the portal sent in, and `X-Session-Id`, the session the backend started for them. The backend must return only that user's applications, and answer `404` for anyone else's. A `401` on any call means the session has ended: the app shows Session Expired.
- **JSON:** camelCase in both directions.
- **Not found:** `404` means not found wherever a route below says "or `404`". Any other failure status is treated as an error.

Entry

The portal opens the app at `/unotp/entry?UserId=...&Syscode=...` (or at `/`, or the old `/Home`, with the same query), both values encrypted. The app decrypts each with the portal's decryption service, starts a session, reads the user's menu, keeps all three in the server session, and redirects to the Dashboard, so neither value stays in the address. Every other page needs that session; without one, or once it has ended, the partner sees Session Expired. A refused entry shows Unauthorized.

Method	Route	Body	Returns
POST	<code>external/decrypt/decrypt</code> (or <code>Backend:External:Decrypt</code>)	<code>{ value }</code>	<code>{ value }</code> in plain text, or <code>400</code> when it cannot be decrypted
POST	<code>sessions</code>	<code>{ userId, sysCode }</code>	<code>UserSession: sessionId, userId, expiresAt</code> . <code>401</code> , <code>403</code> or <code>404</code> when the user or system code is refused.
GET	<code>menu</code>		<code>MenuItem[]</code> (<code>key</code> , <code>name</code>) for the session's user. <code>key</code> is a console feature: <code>new-fd</code> , <code>pis</code> , <code>view-app</code> , <code>short-url</code> , <code>app-status</code> , <code>renew</code> or <code>admin</code> . A feature the menu leaves out is closed, and its address shows Unauthorized. An empty menu refuses entry.

Lists, rules and the partner

The pages hold no data of their own: every list they offer, every limit they check, and who is signed in come from these routes. The lists and the rules are kept for `Backend:ReferenceCacheMinutes` (default 10) before they are asked for again; a failed answer is not kept.

Method	Route	Body	Returns
GET	<code>reference</code>		<code>ReferenceData</code> : every list the pages offer (see below)
GET	<code>config</code>		<code>AppConfig</code> : the limits and rules the pages check

Method	Route	Body	Returns
GET	me		PartnerProfile: name, code, agencyType, brokerCode of the signed-in partner. The sourcingAgency in config chooses the sourcing mode, broker code and deposit category; any other type sources as a broker under brokerCode, with the category set from the holder's date of birth and gender. While Features:DemoData is on, ? agency=2001&broker=BR10874 shows the app as another kind of partner for the session.
POST	deposits/quote	{ amount, tenureMonths, payout, category, startsOn? }	DepositQuote: rate, interestEach, maturityAmount, maturesOn, rateAsOn
GET	ifsc/{code}		BankBranch: ifsc, bank, branch, micr, or 404
GET	ifsc?q={text}		BankBranch[] : the branches whose bank name, branch, IFSC or MICR holds every word of the text, best first, at most 20 — the bank search on Bank Details & Payment
GET	demo/cases		DemoCases: dob, cases ({ pan, dob, folios, shows } ; an empty dob is none on record, no folios a new investor) and notes, for the Test data card on Investor Identification while Features:DemoData is on. A live backend answers 404 and the card is not shown.
GET	demo/banks		DemoBanks: branches (BankBranch[]) and notes, for the Test data card on Bank Details & Payment while Features:DemoData is on. A live backend answers 404 and the card is not shown.

- **ReferenceData:** applicationTypes and renewInstructions and deliveryTypes as { code, name }; categories as { code, name, employee, women, senior }; paymentModes as { name, document } (document is the instrument a copy is filed for, or null); sourcingModes as { code, name, codeLabel, nameLabel, house, search, register, sub, categories }; proofsOfAddress as { type, issuer, hasPhoto } (only a proof with hasPhoto - an officially valid document - proves the permanent address; one without, a utility bill, proves only the communication address); payouts as { code, name, perYear, each } (perYear 0 is cumulative); tenures in months; requiredDocuments as { title, items, notes }; and plain lists for employeeHolders, employeeRelations, employeeProofs, incomeBands, occupations, subOccupations, maritalStatuses, genders, nameTypes, nomineeRelations, cmsLocations, identificationNotes, dashboardNotes, noticeKinds (for Console Admin) and declarations (signed on Review Summary before submitting). Codes are what the app posts and saves.
- **AppConfig:** sourcingAgency (the agency type that chooses how an application is sourced), minAge, seniorAge, maxJointHolders, maxAttempts, minAmount, maxAmount, amountStep (rupees), cancellationDays, draftDays, and linkValidityHours by purpose (payment, acceptance). The backend checks the same rules again on save.

- **Quote:** the rate is the card rate for the tenure and category, locked when the application is submitted. The mock compounds a cumulative deposit half-yearly and pays simple interest per period otherwise.

Investors

Method	Route	Body	Returns
GET	investors/folios?pan={pan}		FolioRecord[] . Every folio held against the PAN; more than one is a record Operations has to merge.
GET	investors/folios/{folio}		FolioRecord , or 404

FolioRecord: pan, dob (dd-MM-yyyy, or "" when none is held), folio, name, gender, address ("" when none is held), docs { pan, photo, poa } (booleans), note.

Applications

Method	Route	Body	Returns
POST	applications	{ holder }	Application . The backend mints the application number.
GET	applications/{appNo}		Application , or 404. Drafts included.
PUT	applications/{appNo}/upload	UploadState , with header If-Match: "{version}"	{ version } , or 409 / 412 if the application changed since that version was read. Nothing is saved in that case.
PUT	applications/{appNo}/details	ApplicationDetails , with If-Match	{ version } , or 409 / 412
PUT	applications/{appNo}/pages/{page}	{ state }	204 , or 404 when the application is not the partner's. A wizard page's working state, kept in pages on the application as it stands: it moves no version and meets no conflict. Opaque to the backend.
PUT	applications/{appNo}/payment	PaymentDetails , with If-Match	{ version } , or 409 / 412
PUT	applications/{appNo}/deposit	DepositDetails , with If-Match	{ version } , or 409 / 412
POST	applications/{appNo}/submit	with If-Match	Application as submitted, or 409 / 412 . Sends the investor the payment link by SMS and e-mail both.
POST	applications/{appNo}/resend-link		Submission , or 404 / 409 when there is no link to resend
GET	applications/drafts		DraftSummary[] : the partner's own applications, opened and not yet submitted, the one touched last first, with identifiers masked

Method	Route	Body	Returns
GET	applications		ApplicationRecord[] : the partner's applications, including older ones, each with its scheme name and milestones ({ step, at }, in order) for the View Application timeline

- **holder**: pan, dob, name, folio, panFiled, address, onRecord { pan, photo, poa } or null, and gender (the folio's, or "").
- **Application**: appNo, holder, version, upload (an UploadState, or null until first saved), prior (attempts on record from before the upload step, newest first, as LogEntry[]), and pages (each wizard page's working state by page name, as the page last saved it).
- **ApplicationDetails**: Investor Information: holders (one per holder type: holder 01/02/03, gender, nameType, parentName, annualIncome, occupation, subOccupation, maritalStatus, mobile, email, fatcaTaxResident, fatcaPermanentResident, pep, pepRelated) and nominee (name, dob, relation, guardianName, guardianLine1-3, guardianPinCode, guardianCity) or null.
- **PaymentDetails**: payment and repayment as { ifsc, accountNumber }, repaymentSameAsPayment, and cheque as { number, date, cmsLocation } or null.
- **DepositDetails**: amount, tenureMonths, payout, autoRenewal, renewInstruction, noTds, deliveryType, as the reference lists code them.
- **Submission**: at, status, linkSentTo (the mobile, masked), linkValidUntil, resendsLeft, linkEmailTo (the e-mail, masked; empty when there is none). Application carries it as submitted, with details, payment and deposit.
- **UploadState**: the upload step as a whole. That covers the choices made (appType, poaType, payMode, sourcing, sourceCode, subBroker, category, the emp* fields, formNo, typedFormNo, ckyc, savedAt, and gender as read off an Aadhaar for a holder the folio gives none for), plus docs, attempts, reads and log. See Applications.cs for every field. It never contains a file or an Aadhaar number.
- **Joint holders**: joint maps a holder type (02 the second holder, 03 the third) to { holder, poaType }, added on Investor Information. Their documents sit in docs, attempts and reads under keys that begin with it (h02-pan, h03-poa). A log entry carries holder (02, 03) when it is a joint holder's, and removed once that holder is taken off. CKYC (ckyc) is fetched for the investor only, so it never takes a joint holder's photograph or proof of address off.

Deposit categories: PUBLIC/GENERAL, WOMEN, SR CITIZEN and SR CITIZEN WOMEN (60 or over), plus EMPLOYEE and EMPLOYEE WOMEN, which only a 1033 partner can book, under MFL-EX.

Documents (DMS)

Method	Route	Body	Returns
POST	applications/{appNo}/documents/{holder}/{slot}	multipart file	2xx. Files the copy against the holder's slot.
DELETE	applications/{appNo}/documents/{holder}/{slot}		2xx, or 404 if there is no copy (the app treats that as done)
POST	applications/{appNo}/documents/{holder}/{slot}/refused	multipart file	{ ref, keptUntil }. Keeps a refused copy aside for analysis, off the application.
GET	applications/{appNo}/documents/{holder}/{slot}		The copy, with Content-Type and Content-Disposition, or 404

`{holder}` is the holder type DMS files under:

Code	Holder	Slots
00	Not holder-specific	form, payment, empproof
01	Investor	pan, photo, poa
02	Second holder	pan, photo, poa
03	Third holder	pan, photo, poa

A holder never changes type: the second holder can only be removed once there is no third, so a third never becomes the second.

A slot holds one copy:

- **Re-upload:** the app deletes the old copy first, then files the new one. This happens only once the new copy has passed its checks; a refused copy leaves the old one in place.
- **Dropped document:** when the partner's choices take a document off the application (for example, switching to a payment mode that has no instrument), the app deletes its copy once the save goes through.

Registers and lists

Method	Route	Returns
GET	sourcing/brokers	Party[] (code, name)
GET	sourcing/staff	Party[] . Includes the partner at the keyboard.
GET	payin-slips	SlipRecord[] : every application paying by cheque or DD, cancelled ones included
GET	links	SentLinkRecord[] : links sent to investors, each by SMS and e-mail, with the masked mobile and email it went to. The link itself is never returned.
GET	links/pending	PendingRecord[] : applications waiting on the investor (appNo, investor, applied, masked mobile and email, due)
GET	console/schedule	{ windows: WindowRecord[], announcements: AnnouncementRecord[] }

Each application in these lists carries its `applied` date. The app counts the cancellation window (`cancellationDays` in `config`) from that date. A `SlipRecord` carries `acceptedOn` once the investor accepts a digital application. Field lists are in `Registers.cs`.

Actions on the lists

Method	Route	Body	Returns
POST	payin-slips/{appNo}		SlipRecord with its new <code>slipNo</code> : the slip, or a fresh one for a reprint. 404 when the application is not found, is cancelled, or is digital and not yet accepted.

Method	Route	Body	Returns
POST	links	{ appNo, purpose }	SentLinkRecord: purpose is payment or acceptance. It goes to the investor's mobile and e-mail both. Any link sent before stops working. The validity is linkValidityHours for the purpose. 404 when the application cannot carry that link.
POST	console/windows	{ features, from, to, notice }	WindowRecord. The backend mints the id and records setBy and setOn. An empty notice leaves partners untold.
POST	console/announcements	{ kind, title, at, detail }	AnnouncementRecord. kind is one of noticeKinds in reference.
POST	console/windows/{id}/end		204. Ends a window that is on, or cancels one still to come, with its notice. 404 when there is none.
DELETE	console/announcements/{id}		204. 404 when there is none.

The page checks each action before sending it (a tile picked, from in the future and before to, a heading), and the backend checks again.

Dashboard

The dashboard asks for its lists all at once and counts the rows itself; the backend has no route that returns counts. A list is asked for only while the user's menu opens its feature. A list that fails is left out and the page still loads. The code is in DashboardController.cs.

Section	Route	Feature	What the dashboard shows
Waiting on you: Close to auto-cancel	GET applications, GET config	view-app	Applications not booked or cancelled with 0 to 3 days left, where days left is cancellationDays minus the days since applied. The soonest is named.
Waiting on you: Waiting on the investor	GET links/pending	short-url	Every row, split by due into "to pay" and "to accept".
Waiting on you: Pay-in slips to generate	GET payin-slips	pis	Rows whose state is pending, on paper or accepted by the investor (digital false, or accepted true).
Continue an incomplete application	GET applications/drafts	new-fd	The latest 3 drafts, each opening on Upload Documents; the rest are on Investor Identification.
Tiles and the bell	GET console/schedule		Tiles switched off for a window, and the notices in the bell.
Top bar	GET me		The partner's name.

Example answers, as the mock gives them (identifiers masked by the backend):

GET applications

```
[{ "appNo": "FBBMFL26F18317", "folio": "MF0044146", "investor": "Ra..... Sh.....",
  "pan": "RASH.....P.....", "amount": 250000, "cumulative": true, "months": 36,
  "payout": "On maturity", "holders": 1, "applied": "2026-09-14T00:00:00",
  "digital": true, "instrument": "Cheque", "branch": "Pune Camp",
  "state": "awaiting", "fdr": null, "step": "Payment", "scheme": "Samruddhi",
  "milestones": [{ "step": "Application raised", "at": "2026-09-14T00:00:00" },
    { "step": "Payment received", "at": null } ] }
```

GET config

```
{ "sourcingAgency": "1033", "minAge": 18, "seniorAge": 60, "maxJointHolders": 2,
  "maxAttempts": 3, "minAmount": 5000, "maxAmount": 20000000, "amountStep": 1000,
  "cancellationDays": 14, "draftDays": 30,
  "linkValidityHours": { "payment": 48, "acceptance": 72 } }
```

GET links/pending

```
[{ "appNo": "FBBMFL26F20488", "investor": "An..... Jo.....",
  "applied": "2026-09-25T00:00:00", "mobile": "90.....1000",
  "due": "payment", "email": "a.....@gmail.com" }]
```

GET payin-slips

```
[{ "appNo": "FBBMFL26F16421", "investor": "Su..... Pa.....", "amount": 150000,
  "instrument": "Cheque", "instrumentNo": "104139", "drawnOn": "HDFC Bank",
  "applied": "2026-09-23T00:00:00", "branch": "Nashik Road", "digital": true,
  "accepted": true, "state": "pending", "slipNo": null,
  "acceptedOn": "2026-09-24T00:00:00" }]
```

GET applications/drafts

```
[{ "appNo": "FBBMFL26F10421", "name": "P..... K.....", "pan": "ABCPK.....F",
  "dob": "..../1984", "amount": 100000 }]
```

Outside services

The app asks each check separately, in this order: identification, OCR, then whoever answers for what was read. Only after that does it file the copy with DMS. The name and date of birth OCR reads off a proof of address are matched with the holder's (a name with an initial or a word left out partly matches; a utility bill prints no date of birth). Once both a holder's PAN copy and their proof of address are filed, the faces on them are compared; for now both answers are only shown, and the proof stays filed whatever they say. A holder's PAN copy is taken before their proof of address, and the PAN–Aadhaar link card appears only once an Aadhaar is filed as a proof.

- **An Aadhaar is taken on its name and date of birth.** The name and date of birth OCR reads off an Aadhaar (as proof of address or communication address, for the investor or a joint holder) must match the PAN's: the name as NSDL or the folio holds it, allowing initials or a name left out, and the date of birth exactly. If either differs or can't be read, the copy is refused, counts as an attempt, and is kept aside like any other refusal. A masked Aadhaar is accepted. The masking service is not called by the upload step.

- **An Aadhaar number OCR can't read is typed.** When an Aadhaar is filed for a holder with no folio and OCR reads no whole 12-digit number, the partner types it for the PAN–Aadhaar link. It must be 12 digits, not start with 0 or 1, and pass the Verhoeff check digit. Like a number read, it is kept in the server session only and never sent to the backend, except to the link check.

Service	Interface	With Idfy.Api	Without IDfy (external/{name}/)
NSDL	INsdlService	Not covered by IDfy	POST verify { pan, dob, name } → { pairOk, nameOk }
Identification	IDocumentIdentifier	POST /api/documents/validate: with docType for a PAN; with no docType for a proof of address, whose detected_doc_type says which proof it is (Aadhaar, passport, driving licence or voter ID)	POST identify (multipart file, expected, type) → { matches, hint, type }. Used for a cheque, and for a proof of address IDfy does not know (a utility bill). For a proof of address type is sent empty and the answer's type says which proof it is: Aadhaar, Passport, Driving Licence, Voter ID or Utility bill. That becomes the proof's type on the application: it is never chosen.
Masking	IMaskingService	POST /api/aadhaar/mask. A copy with no number to mask (id_number_found: false) is already masked.	POST check (multipart file, consent) → { masked }. Not called by the upload step, which relies on OCR reading the whole number instead.
OCR	IOcrService	/api/pan/extract, /api/ aadhaar/extract (QR code read first), /api/driving-license/extract (expiry from date_of_validity and validity, leaving out any date that is an issue_dates date or not after the latest one), /api/passport/ extract, /api/voter-id/extract	POST read (multipart file, kind, type, pan, dob, name, consent) → OcrReading (a PAN card's carries pan and dob as dd-MM-yyyy, which must match the holder's or the copy is refused; an Aadhaar's also carries gender; a cheque's carries cheque - { accountNumber, ifsc, micr, number, date }, the date as dd-MM- yyyy - which Bank Details is filled in from once the bank confirms the account). Used for a utility bill or a cheque.
Verification	IVerificationService	/api/driving-license/verify/ sync and /api/passport/verify/ sync (both with the holder's date of birth), /api/voter-id/verify/sync. id_found counts as confirmed. For a licence, the later of nt_validity_to and t_validity_to is returned as expiry, and dl_status as standing.	POST proof { proofType, reading, holderDob } and POST account { account, bank } → { confirmed, verifier, notAsked, expiry, standing } (expiry as dd-MM-yyyy, or empty; once confirmed it replaces the expiry OCR read off the copy). Used for an Aadhaar (IDfy has no UIDAI source check) and for bank accounts.
PAN–Aadhaar link	IPanAadhaarLinkService	/api/pan-aadhaar-link/verify/ sync	POST check { pan, aadhaarNumber } → { link }
PAN–POA face match	IFaceMatchService	POST /api/face/compare with document (the PAN copy) and document2 (the proof of address) as Base64, each 150–4,096 px a side; reads is_a_match, match_score, review_recommended and image_1/image_2.face_detected and face_quality. No face found, or a review recommended, is shown as "Not sure"	POST compare (multipart pan, proof) → { matched, score, unsure }

What the app expects from every outside service

- **Failures:** a service that is down, slow, answers with an error, or answers with something that can't be read is reported to the partner as "could not answer". Nothing is filed and no refusal is counted. This holds for the `Idfy.Api` client and the `external/{name}/` clients alike.
- **"Not asked" is not a refusal.** Verification returns `notAsked` with a reason when it had nothing to ask the issuer with: a passport file number (it is on the last page), an unreadable licence number, or a voter ID EPIC number that IDfy returned partly masked. The copy is filed and Operations settle the address.
- **The link is asked only for a holder with no folio.** A holder on a folio never has the PAN–Aadhaar link asked, whatever they file.
- **The link check never costs a copy.** If the PAN–Aadhaar link can't be asked, the PAN or Aadhaar that prompted it is still filed. The link is marked "not checked" and asked again when an Aadhaar is next filed.
- **Aadhaar number:** only a whole 12-digit number is used for the link check: the one OCR reads, or the one the partner types when OCR can't read it. A masked number, or the partial one a secure QR code carries, is ignored.

Rules the app keeps when calling Idfy.Api

- **Images:** sent as Base64 in JSON rather than to the `/upload` routes, because those routes accept only `image/*` and a proof can be a PDF. Anything over the 3,000,000-character Base64 cap is refused before it is sent.
- **Results:** a task is accepted only when its `status` is `completed`.
- **Retries:** nothing is retried, since a repeated call can be charged twice.
- **Errors:** problem+json errors are logged with their `traceId`, which is also shown in the upload history. A failed check files nothing and doesn't count against the three attempts.
- **Aadhaar consent:** Aadhaar calls are sent only with the holder's consent. The upload step doesn't ask for it, so behind a real IDfy an Aadhaar is turned back with a message saying why. The mock doesn't ask for consent.
- **The link check needs both numbers:** it runs when the second of the PAN and the Aadhaar number arrives. The Aadhaar number, read by OCR or typed, is kept only in the server session for that application. It is never saved to the backend.